EN3160 - Assignment 02: Fitting and Alignment

Name: Ahamed A.M.S
Index number: 230020R
GitHub profile: https://github.com/shaahidahamed

In [12]:
```
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
from skimage.feature import blob_log
from matplotlib.patches import Circle
from scipy.optimize import least_squares
```

In [4]:
```
# Load image
img = cv.imread('a2images/the_berry_farms_sunflower_field.jpeg',
                cv.IMREAD_REDUCED_COLOR_4)

# Convert to grayscale
gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
gray_inv = cv.bitwise_not(gray)

# Sigma range
min_sigma = 1
max_sigma = 30

# Detect bright and dark blobs using LoG
bright_blobs = blob_log(gray, min_sigma=min_sigma,
                        max_sigma=max_sigma,
                        num_sigma=15, threshold=0.1)

dark_blobs = blob_log(gray_inv, min_sigma=min_sigma,
                      max_sigma=max_sigma,
                      num_sigma=15, threshold=0.1)


# Combine all blobs
blobs = np.vstack((bright_blobs, dark_blobs))

# Keep only circles completely inside the image
h, w = gray.shape
valid_blobs = []

for y, x, sigma in blobs:
    radius = np.sqrt(2) * sigma

    if radius <= x < w-radius and radius <= y < h-radius:
        valid_blobs.append((y, x, sigma))

# Sort valid blobs by size
```

```python
largest_blobs = sorted(valid_blobs, key=lambda b: b[2], reverse=True)

result = img.copy()

# Draw all detected blobs in blue
for y, x, sigma in blobs:
    radius = round(np.sqrt(2) * sigma)
    cv.circle(result, (round(x), round(y)),
              radius, (255, 0, 0), 1)

# Highlight five largest complete blobs in green
for y, x, sigma in largest_blobs[:5]:
    radius = round(np.sqrt(2) * sigma)
    cv.circle(result, (round(x), round(y)),
              radius, (0, 255, 0), 2)

# Display results
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

axes[0].imshow(cv.cvtColor(img, cv.COLOR_BGR2RGB))
axes[0].set_title("Original Image")

axes[1].imshow(cv.cvtColor(result, cv.COLOR_BGR2RGB))
axes[1].set_title("LoG Detected Blobs")

for ax in axes:
    ax.axis('off')

plt.tight_layout()
plt.show()

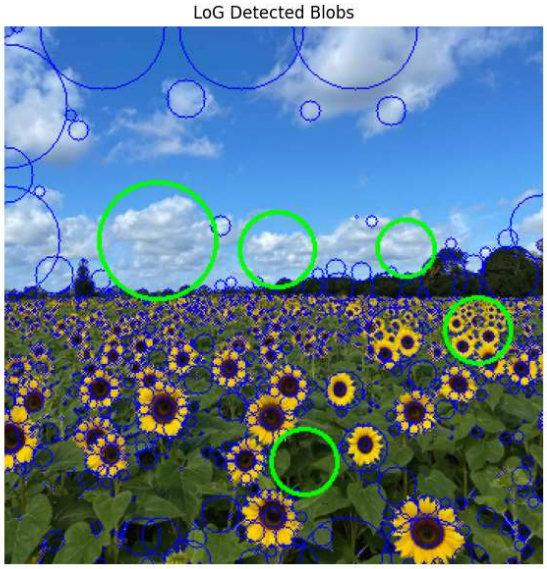
# Report results
print("Sigma range:", min_sigma, "to", max_sigma)
print("Total blob detections:", len(blobs))
print("Complete blobs:", len(valid_blobs))

print("\nFive Largest Complete Circles:")

for i, (y, x, sigma) in enumerate(largest_blobs[:5], 1):
    radius = np.sqrt(2) * sigma

    print(f"Circle {i}: Center=({x:.2f}, {y:.2f}), "
          f"Radius={radius:.2f}, Sigma={sigma:.2f}")
```






```
Sigma range: 1 to 30
Total blob detections: 1496
Complete blobs: 1446

Five Largest Complete Circles:
Circle 1: Center=(102.00, 143.00), Radius=39.50, Sigma=27.93
Circle 2: Center=(182.00, 149.00), Radius=24.85, Sigma=17.57
Circle 3: Center=(316.00, 203.00), Radius=21.92, Sigma=15.50
Circle 4: Center=(200.00, 291.00), Radius=21.92, Sigma=15.50
Circle 5: Center=(268.00, 148.00), Radius=18.99, Sigma=13.43
```

LoG was used to detect blobs using sigma values from 1 to 30 with 15 scales. A total of 1,496 blobs were detected, and 1,446 were fully inside the image. The largest circle had a radius of 39.50 pixels.

The method detected many circular regions, but some were from trees and background areas rather than sunflowers. This is because LoG detects blobs based on intensity differences. Further filtering could help detect sunflower centers more accurately.

Question 2: Manual RANSAC, line then circle

In [5]:
```
np.random.seed(0)
N = 100
half_n = N//2
r = 10
x0_gt , y0_gt = 2, 3 # Center
s = r/16
t = np.random.uniform(0, 2*np.pi , half_n)
n = s*np.random.randn(half_n)
x, y = x0_gt + (r + n)*np.cos(t), y0_gt + (r + n)*np.sin(t)
X_circ = np.hstack((x.reshape(half_n ,1) , y.reshape(half_n ,1)))
s = 1.
m, b = -1, 2
x = np.linspace(-12, 12, half_n)
y = m*x + b + s*np.random.randn(half_n)
```

```
X_line = np.hstack((x.reshape(half_n ,1) , y.reshape(half_n ,1)))
X = np.vstack((X_circ , X_line )) # All points
```

In [6]:

```
# Fit line using Total Least Squares
def fit_line(points):
    center = points.mean(axis=0)

    _, _, V = np.linalg.svd(points - center)
    a, b = V[-1]

    d = a*center[0] + b*center[1]

    if d < 0:
        a, b, d = -a, -b, -d

    return np.array([a, b, d])


# Perpendicular distance to line
def line_error(points, model):
    a, b, d = model
    return np.abs(a*points[:, 0] + b*points[:, 1] - d)


# Fit circle through three points
def fit_circle(points):
    p1, p2, p3 = points

    A = 2 * np.array([p2-p1, p3-p1])
    B = np.array([p2@p2-p1@p1, p3@p3-p1@p1])

    center = np.linalg.solve(A, B)
    radius = np.linalg.norm(p1-center)

    return np.array([center[0], center[1], radius])


# Radial distance error
def circle_error(points, model):
    cx, cy, radius = model
    distances = np.linalg.norm(points - [cx, cy], axis=1)
    return np.abs(distances - radius)


# Manual RANSAC
def ransac(points, sample_size, fit_func, error_func,
           threshold, min_inliers, seed):

    rng = np.random.default_rng(seed)

    best_count = 0
    best_error = np.inf
    best = None

    for i in range(2000):
```

```
        ids = rng.choice(len(points), sample_size, replace=False)
        model = fit_func(points[ids])

        errors = error_func(points, model)
        inliers = errors < threshold

        count = inliers.sum()
        total_error = np.sum(errors[inliers]**2)

        if count >= min_inliers and (
            count > best_count or
            (count == best_count and total_error < best_error)
        ):
            best_count = count
            best_error = total_error

            best = {
                'sample': ids,
                'sample_model': model,
                'inliers': inliers
            }

    return best
```

In [7]:
```
# Part (a): Fit line using RANSAC
line = ransac(X, 2, fit_line, line_error,
              1.5, 35, 42)

# Refine line using its inliers
line_model = fit_line(X[line['inliers']])
line_mask = line_error(X, line_model) < 1.5

# Part (b): Remove line inliers
remaining_ids = np.where(~line_mask)[0]
remaining = X[remaining_ids]

# Fit circle using RANSAC
circle = ransac(remaining, 3, fit_circle, circle_error,
                1.25, 25, 43)

# Refine circle using least squares
circle_points = remaining[circle['inliers']]

result = least_squares(
    lambda p: np.linalg.norm(circle_points-p[:2], axis=1)-p[2],
    circle['sample_model'],
    bounds=([-np.inf, -np.inf, 1e-8], [np.inf, np.inf, np.inf])
)

circle_model = result.x

# Final circle inliers
circle_mask = np.zeros(len(X), dtype=bool)
circle_mask[remaining_ids] = circle_error(remaining, circle_model) < 1.25

# Display parameters
```

```
print("Line parameters (a, b, d):", np.round(line_model, 3))
print("Normal magnitude:", np.linalg.norm(line_model[:2]))
print("Line inliers:", line_mask.sum())

print("\nCircle parameters (cx, cy, r):", np.round(circle_model, 3))
print("Circle inliers:", circle_mask.sum())

print("\nLine RMS error:",
      np.sqrt(np.mean(line_error(X[line_mask], line_model)**2)))

print("Circle RMS error:",
      np.sqrt(np.mean(circle_error(X[circle_mask], circle_model)**2)))
```

```
Line parameters (a, b, d): [0.716 0.698 1.562]
Normal magnitude: 0.9999999999999999
Line inliers: 54

Circle parameters (cx, cy, r): [1.949 2.988 9.892]
Circle inliers: 44

Line RMS error: 0.6754616681448892
Circle RMS error: 0.5358998310131629
```

In [8]:

```
fig, ax = plt.subplots(figsize=(8, 8))

# All points
ax.scatter(X[:, 0], X[:, 1], color='gray',
           s=15, label='All points')

# Line inliers
ax.scatter(X[line_mask, 0], X[line_mask, 1],
           color='orange', s=18, label='Line inliers')

# Circle inliers
ax.scatter(X[circle_mask, 0], X[circle_mask, 1],
           color='cyan', s=18, label='Circle inliers')

# Best line sample
ax.scatter(X[line['sample'], 0], X[line['sample'], 1],
           color='red', marker='x', s=75,
           label='Line sample', zorder=5)

# Best circle sample
ax.scatter(remaining[circle['sample'], 0],
           remaining[circle['sample'], 1],
           color='blue', marker='x', s=75,
           label='Circle sample', zorder=5)


# Draw line function
def draw_line(model, color, style, label):
    a, b, d = model

    center = np.array([a, b]) * d
    direction = np.array([-b, a])
```

```
    pts = center + np.array([-25, 25])[:, None] * direction

    ax.plot(pts[:, 0], pts[:, 1],
            color=color, linestyle=style, label=label)


# Estimated lines
draw_line(line_model, 'red', '-', 'Best-fit line')
draw_line(line['sample_model'], 'orange', ':', 'Sample line')

# Estimated circles
ax.add_patch(Circle(
    circle_model[:2], circle_model[2],
    fill=False, color='blue', linewidth=2,
    label='Best-fit circle'
))

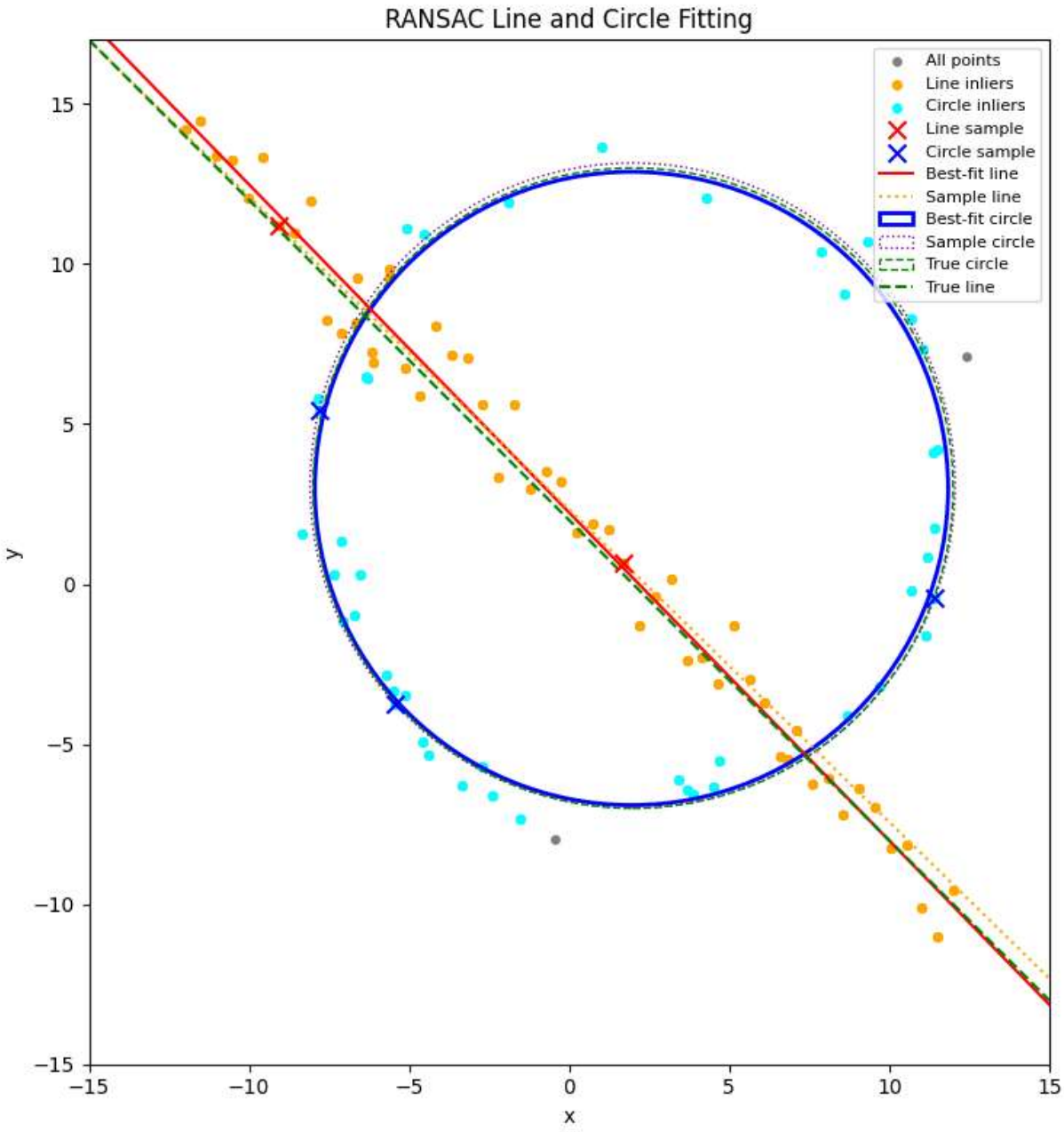
ax.add_patch(Circle(
    circle['sample_model'][:2],
    circle['sample_model'][2],
    fill=False, color='purple', linestyle=':',
    label='Sample circle'
))

# Ground truth circle
ax.add_patch(Circle(
    (x0_gt, y0_gt), r,
    fill=False, color='green', linestyle='--',
    label='True circle'
))

# Ground truth line
x_plot = np.array([-15, 15])
ax.plot(x_plot, m*x_plot+b, 'g--', label='True line')

ax.set_xlim(-15, 15)
ax.set_ylim(-15, 17)
ax.set_aspect('equal')
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_title('RANSAC Line and Circle Fitting')
ax.legend(fontsize=8, loc='upper right')

plt.tight_layout()
plt.show()
```



(d) What will happen if we fit the circle first?

In [9]:

```
# Fit circle before removing line points
circle_first = ransac(
    X, 3, fit_circle, circle_error,
    1.25, 25, 44
)

print("Circle-first parameters:",
      np.round(circle_first['sample_model'], 3))

print("Total circle-first inliers:",
      circle_first['inliers'].sum())

print("Actual circle points selected:",
      circle_first['inliers'][:half_n].sum())
```

```
print("Line points incorrectly selected:",
      circle_first['inliers'][half_n:].sum())
```

```
Circle-first parameters: [ 1.992  3.088 10.012]
Total circle-first inliers: 58
Actual circle points selected: 49
Line points incorrectly selected: 9
```

When the circle is fitted first, some points belonging to the line may also be identified as circle inliers because they fall within the radial error threshold. This can affect the accuracy of the estimated circle and leave fewer points for subsequent line fitting. However, fitting the circle first does not always fail. In this experiment, the correct circle was approximately identified, but some line points were included in its consensus. Therefore, fitting the line first and removing its inliers helps reduce interference during circle fitting.

Question 3: planar homography and blending

In [10]:
```
# Select four points using mouse
def select_points(image):
    points = []
    display = image.copy()

    def click(event, x, y, flags, param):
        if event == cv.EVENT_LBUTTONDOWN and len(points) < 4:
            points.append([x, y])
            cv.circle(display, (x, y), 5, (0, 0, 255), -1)

    cv.namedWindow("Select Points", cv.WINDOW_NORMAL)
    cv.setMouseCallback("Select Points", click)

    print("Click 4 corners: TL, TR, BR, BL")

    while len(points) < 4:
        cv.imshow("Select Points", display)
        cv.waitKey(20)

    cv.destroyWindow("Select Points")
    return np.float32(points)

# Calculate homography and blend images
def apply_homography(background, overlay, points, alpha):

    h, w = overlay.shape[:2]
    bg_h, bg_w = background.shape[:2]

    # Overlay image corners
    src = np.float32([
        [0, 0], [w-1, 0],
        [w-1, h-1], [0, h-1]
    ])

    # Homography
```

```
    H, _ = cv.findHomography(src, points)

    # Warp overlay
    warped = cv.warpPerspective(overlay, H, (bg_w, bg_h))

    # Create mask
    mask = np.ones((h, w), dtype=np.uint8) * 255
    mask = cv.warpPerspective(mask, H, (bg_w, bg_h))

    # Blend images
    blended = cv.addWeighted(background, 1-alpha, warped, alpha, 0)

    final = background.copy()
    final[mask > 0] = blended[mask > 0]

    return final, H

# Display results
def show_result(background, overlay, final, points):

    fig, axes = plt.subplots(1, 3, figsize=(18, 6))

    axes[0].imshow(cv.cvtColor(background, cv.COLOR_BGR2RGB))
    for x, y in points:
        axes[0].plot(x, y, 'ro')
    axes[0].set_title("Selected Points")

    axes[1].imshow(cv.cvtColor(overlay, cv.COLOR_BGR2RGB))
    axes[1].set_title("Overlay Image")

    axes[2].imshow(cv.cvtColor(final, cv.COLOR_BGR2RGB))
    axes[2].set_title("Final Image")

    for ax in axes:
        ax.axis('off')

    plt.tight_layout()
    plt.show()

# Image pairs
image_pairs = [
    ('a2images/wadham.jpg', 'a2images/flag.png', 0.6),
    ('a2images/home_theater_screens.jpg', 'a2images/emma.jpg', 0.6),
    ('a2images/wall.jpg', 'a2images/jeniffer.jpg', 0.8),
    ('a2images/stadium.jpg', 'a2images/spider.png', 0.9)]

# Process all image pairs
for i, (bg_file, overlay_file, alpha) in enumerate(image_pairs, 1):

    background = cv.imread(bg_file)
    overlay = cv.imread(overlay_file)

    print(f"\nProcessing Pair {i}")

    # Select destination points
    points = select_points(background)
```

```
    # Compute homography and blend
    final, H = apply_homography(background, overlay, points, alpha)

    # Plot results
    show_result(background, overlay, final, points)

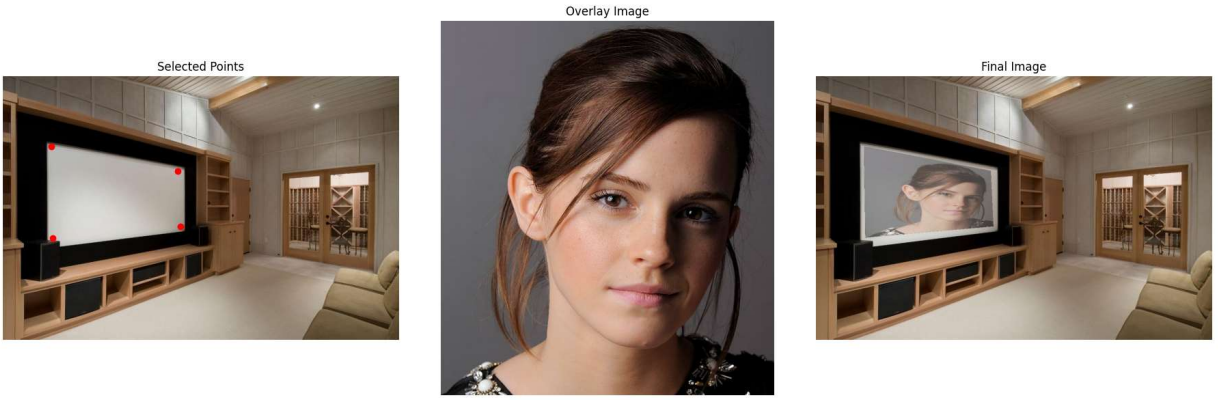
    print("Homography Matrix:\n", H)
```

```
Processing Pair 1
Click 4 corners: TL, TR, BR, BL
```



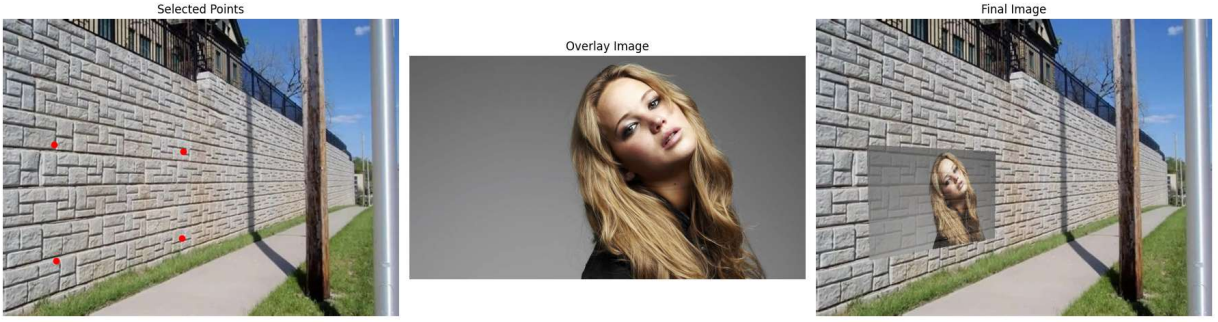

```
Homography Matrix:
 [[ 2.71931018e-01 -2.30711988e-02  1.38000000e+02]
 [ 9.25177042e-02  1.53859848e-01  3.25000000e+02]
 [ 1.73528535e-04 -9.87786728e-05  1.00000000e+00]]

Processing Pair 2
Click 4 corners: TL, TR, BR, BL
```




```
Homography Matrix:
 [[ 1.00215951e+00  5.82225845e-04  1.46000000e+02]
 [ 3.58514043e-01  3.26912239e-01  2.14000000e+02]
 [ 8.87476803e-04 -2.90809995e-05  1.00000000e+00]]

Processing Pair 3
Click 4 corners: TL, TR, BR, BL
```



```
Homography Matrix:
 [[1.18277104e-01 4.75909499e-03 1.21000000e+02]
 [3.23619905e-02 1.36322899e-01 2.99000000e+02]
 [8.95055190e-05 1.55907922e-05 1.00000000e+00]]

Processing Pair 4
Click 4 corners: TL, TR, BR, BL
```




```
Homography Matrix:
 [[ 3.20291858e-01  1.53528977e-01  1.20000000e+02]
 [ 3.71439950e-02 -7.40388643e-02  2.88000000e+02]
 [ 6.44496055e-04 -6.54251203e-04  1.00000000e+00]]
```

Four image pairs were selected to demonstrate how images can be placed on different surfaces using perspective transformation.

- **Building and flag:** To show how a flag can be placed naturally on a building wall.
- **Home theater and Emma image:** To simulate displaying an image on a theater screen.
- **Wall and Jennifer image:** To demonstrate placing a picture or poster on a wall.
- **Stadium and Spider image:** To show how an image can be displayed on a stadium surface.

These examples were selected because they represent different real-world applications of image superimposition and perspective transformation.

Question 4: PCB alignment and differences

In [11]:
```python
# Align PCB image with template
def align_pcb(image, template):

    # Detect SIFT features
    sift = cv.SIFT_create(nfeatures=500)

    kp1, des1 = sift.detectAndCompute(image, None)
    kp2, des2 = sift.detectAndCompute(template, None)

    # Match features using BFMatcher
    bf = cv.BFMatcher()
    matches = bf.knnMatch(des1, des2, k=2)

    # Select good matches
```

```
    good = []
    for m, n in matches:
        if m.distance < 0.75 * n.distance:
            good.append(m)

    # Get matching points
    src = np.float32([kp1[m.queryIdx].pt for m in good])
    dst = np.float32([kp2[m.trainIdx].pt for m in good])

    # Find homography using RANSAC
    H, mask = cv.findHomography(src, dst, cv.RANSAC, 4.0)

    # Draw matched features
    matched = cv.drawMatches(
        image, kp1, template, kp2,
        good, None,
        matchesMask=mask.ravel().tolist(),
        flags=cv.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
    )

    # Align PCB with template
    h, w = template.shape[:2]
    aligned = cv.warpPerspective(image, H, (w, h))

    # Calculate difference
    difference = cv.absdiff(template, aligned)

    # Remove invalid border areas
    valid = np.ones(image.shape[:2], dtype=np.uint8) * 255
    valid = cv.warpPerspective(valid, H, (w, h))
    difference[valid == 0] = 0

    return matched, aligned, difference, len(good), int(mask.sum())

# Load reference PCB
template = cv.imread('a2images/pcbs/AECC1.jpg')
template = cv.resize(template, None, fx=0.25, fy=0.25)

# PCB images
pcb_images = ['AECC2.jpg','AECC3.jpg','AECC4.jpg','AAAA1.jpg','ADAD1.jpg']

# Process each PCB image
for filename in pcb_images:

    pcb = cv.imread('a2images/pcbs/' + filename)
    pcb = cv.resize(pcb, None, fx=0.25, fy=0.25)

    # SIFT matching and alignment
    matched, aligned, difference, matches, inliers = align_pcb(pcb, template)

    print(f"\n{filename}")
    print("Good matches:", matches)
    print("RANSAC inliers:", inliers)

    # Plot results
    fig, axes = plt.subplots(1, 3, figsize=(15, 5))
```

```
    axes[0].imshow(cv.cvtColor(matched, cv.COLOR_BGR2RGB))
    axes[0].set_title("SIFT Matches")

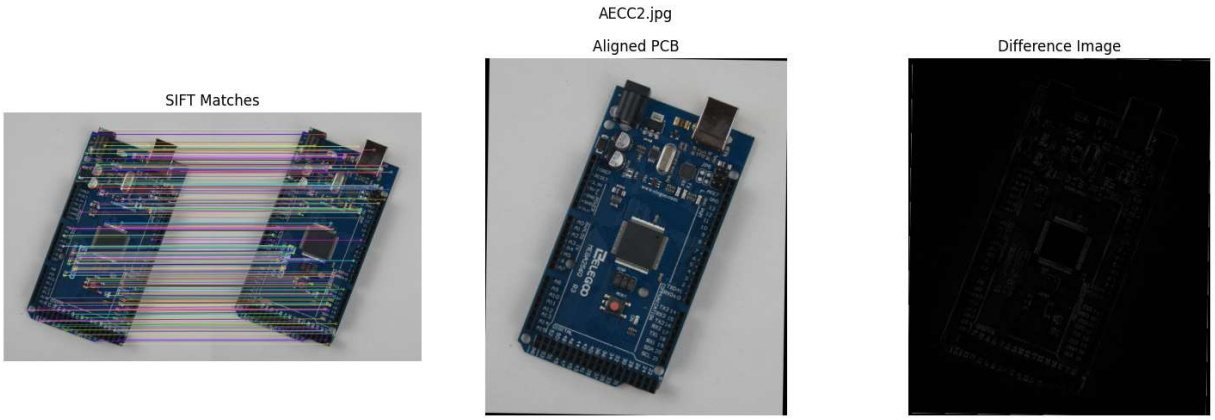
    axes[1].imshow(cv.cvtColor(aligned, cv.COLOR_BGR2RGB))
    axes[1].set_title("Aligned PCB")

    axes[2].imshow(cv.cvtColor(difference, cv.COLOR_BGR2RGB))
    axes[2].set_title("Difference Image")

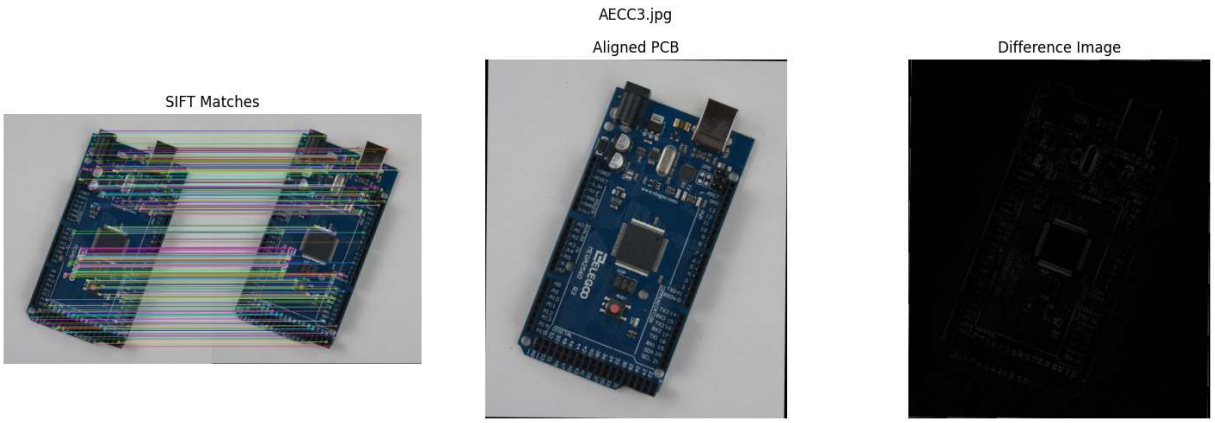
    for ax in axes:
        ax.axis('off')

    plt.suptitle(filename)
    plt.tight_layout()
    plt.show()
    plt.close(fig)
```

AECC2.jpg
Good matches: 321
RANSAC inliers: 305



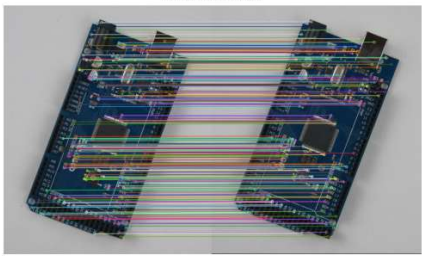
AECC3.jpg
Good matches: 316
RANSAC inliers: 295



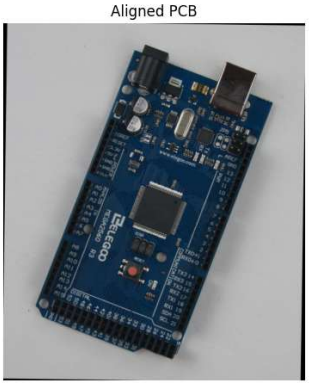
AECC4.jpg
Good matches: 313
RANSAC inliers: 292

AECC4.jpg

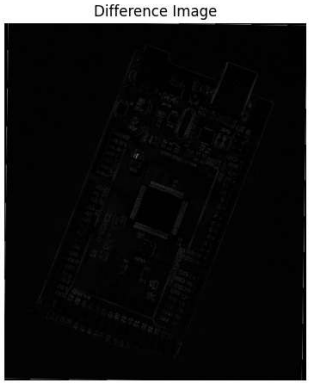
SIFT Matches



Aligned PCB



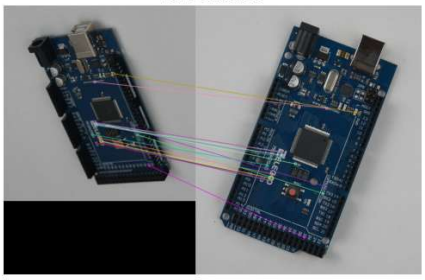
Difference Image



```
AAAA1.jpg
Good matches: 29
RANSAC inliers: 18
```

AAAA1.jpg

SIFT Matches



Aligned PCB



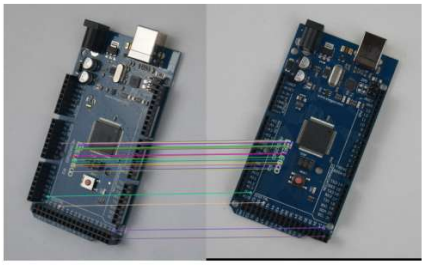
Difference Image



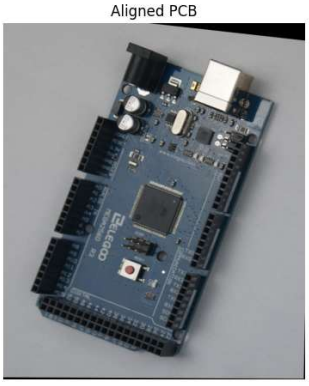
```
ADAD1.jpg
Good matches: 41
RANSAC inliers: 26
```

ADAD1.jpg

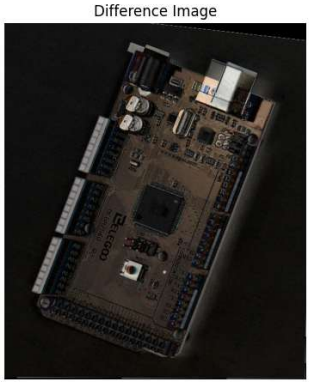
SIFT Matches



Aligned PCB



Difference Image



(e) Some elements such as the USB Type B port, capacitors, and reset button show hig hdifference. What could be the reason for this?

Some components, such as the USB Type-B port, capacitors, and reset button, show high differences because they have different heights above the PCB surface. Homography assumes that the points lie on a single plane. Therefore, these components may not align perfectly due to perspective differences. Changes in lighting, reflections, and shadows can also contribute to the differences.